

Contratos compartidos — BC Importados · Parcial I

(06/10/2026)

Este archivo es la referencia del equipo. **No se pega en la consola:** lo que se pega es el prompt individual de cada uno. Está acá para resolver discusiones sin tener que leer cuatro prompts.

Quién hace qué

Persona	Módulo	Rama	Prefijo de migración
Agustín	Kernel, infraestructura, inventario y pagos	feat/kernel , feat/inventario	20261004_03_
Yasmín	Identidad, acceso y RBAC	feat/identidad	20261004_01_
Genaro	Catálogo	feat/catalogo	20261004_02_
Nicolás	Carrito, checkout y pedidos	feat/pedidos	20261004_04_

El orden de los prefijos es el orden en que corren las migraciones, y respeta las foreign keys: identidad → catálogo → inventario → pedidos.

Línea de tiempo

Momento	Qué tiene que estar
Domingo 04/10, +2 h	Agustín: kernel en <code>develop</code> (<code>main.go</code> levanta, <code>/api/salud</code> responde, <code>errors</code> y <code>router</code> listos, <code>go.mod</code> con todas las dependencias). Hasta que esto no esté, nadie puede compilar handlers.
Domingo 04/10, +5 h	Yasmín: <code>middleware.RequiereSesion</code> y <code>RequierePermiso</code> en <code>develop</code> . Hasta que esto no esté, nadie puede proteger endpoints.
Domingo 04/10, fin del día	Los cuatro: dominio + migración + repositorios mergeados en <code>develop</code> . La base en la nube migrada.
Lunes 05/10	Endpoints completos, Mercado Pago en sandbox, deploy con URL pública, <code>docs/api/*.http</code> corriendo contra producción.
Martes 06/10, mañana	Ensayo: cada uno explica el módulo de otro .

Reglas que valen para los cuatro

- Nadie edita un archivo de otro.** Si necesitás un cambio en el módulo de otra persona, lo pedís por el grupo. Los únicos archivos compartidos son `go.mod`, `go.sum`, `cmd/api/main.go`, `router.go` y `registry.go`, y los cuatro los escribe Agustín en el bloque 0 y después no se tocan.
- La identidad baja de la sesión, nunca del body.** Si `usuarioID`, `rol` o `permiso` aparecen en un DTO, en un header propio o en la query, hay un agujero.
- Toda consulta que devuelve datos de un usuario lleva su id en el WHERE.** No un `if` después de traer la fila.
- Nada de panic para errores esperados.** Se devuelven los errores de `apierr` y el handler los traduce.
- Plata en centavos, int64.** Nunca `float64`. Las columnas son `bigint` y el campo se llama `..._centavos`.

6. **Identificadores** `uuid.UUID`. Nunca autoincremental: no queremos que se adivine la URL del pedido del vecino.
7. **Cada commit es una pieza que compila.** `go build ./...` y `go vet ./...` en verde antes de cada push.

Contrato 1 — Errores HTTP (dueño: Agustín)

Paquete `apierr` en `backend/internal/adapters/rest/errors/`.

```
package apierr

// Cuerpo único de todas las respuestas de error del sistema.
type Cuerpo struct {
    Error    string `json:"error"`
    Detalles any    `json:"detalles,omitempty"`
}

// Errores de dominio. Cualquier capa los devuelve; sólo el handler los traduce.
var (
    ErrNoAutenticado = errors.New("no autenticado") // → 401
    ErrNoAutorizado  = errors.New("no autorizado")  // → 403
    ErrNoEncontrado  = errors.New("no encontrado")  // → 404
)

// Validación de forma: lo decide el body solo.           → 400
type ErrValidacion struct{ Campos map[string]string }

// Regla de negocio: hace falta ir a buscar estado.       → 409
type ErrRegla struct {
    Mensaje string
    Datos   any // el dato estructurado que el criterio de aceptación pide enumerar
}

func JSON(w http.ResponseWriter, status int, cuerpo any)
func Responder(w http.ResponseWriter, endpoint string, err error)
```

`Responder` traduce: `ErrNoAutenticado` → 401, `ErrNoAutorizado` → 403, `ErrNoEncontrado` → 404, `ErrValidacion` → 400 con `detalles.campos`, `ErrRegla` → 409 con sus `Datos`, y cualquier otra cosa → `log.Printf(endpoint, err) + 500 {"error":"Error interno"}` sin detalles.

Contrato 2 — Sesión y permisos (dueño: Yasmín)

Paquete `middleware` en `backend/internal/adapters/rest/middleware/`.

```

package middleware

type Usuario struct {
    ID        uuid.UUID
    Correo    string
    Roles     []string
    Permisos  []string
}

func Nuevo(secreto string) *Middleware

func (m *Middleware) RequiereSesion(next http.Handler) http.Handler
func (m *Middleware) RequierePermiso(permiso string) func(http.Handler) http.Handler
func (m *Middleware) SesionOpcional(next http.Handler) http.Handler // para checkout de invitado

func UsuarioDeContexto(ctx context.Context) (Usuario, bool)

```

Los permisos viajan dentro del JWT. Consecuencia que hay que saber contestar en la defensa: si el dueño le cambia los roles a alguien, ese cambio recién aplica cuando esa persona vuelve a iniciar sesión. Está escrito en [ADR-006](#).

Permisos del sistema (los crea el seed de Yasmín):

```

usuarios.gestionar  catalogo.escribir  stock.gestionar  pedidos.gestionar

```

Contrato 3 — Catálogo para otros módulos (dueño: Genaro)

```

package catalogo // backend/internal/usecases/catalogo

type VarianteVendible struct {
    ID                uuid.UUID
    ProductoID        uuid.UUID
    Codigo            string
    Nombre            string
    PrecioMinoristaCentavos int64
    Activa            bool
}

func NuevoLector(db *gorm.DB) *Lector
func (l *Lector) VariantePorID(ctx context.Context, id uuid.UUID) (VarianteVendible, error)
// devuelve apierr.ErrNoEncontrado si no existe o está inactiva

```

Contrato 4 — Stock para otros módulos (dueño: Agustín)

```
package inventario // backend/internal/usecases/inventario

type ItemReserva struct {
    VarianteID uuid.UUID
    Unidades    int
}

func NuevoServicioStock(db *gorm.DB) *ServicioStock

func (s *ServicioStock) Disponible(ctx context.Context, varianteID uuid.UUID) (int, error)
func (s *ServicioStock) Reservar(ctx context.Context, tx *gorm.DB, pedidoID uuid.UUID, items []ItemReserva) error
func (s *ServicioStock) Confirmar(ctx context.Context, tx *gorm.DB, pedidoID uuid.UUID) error
func (s *ServicioStock) Liberar(ctx context.Context, tx *gorm.DB, pedidoID uuid.UUID) error
```

`Reservar` devuelve `apierr.ErrRegla` con `Datos: map[string]int` (variante → unidades disponibles) cuando no alcanza el stock. Ese mapa es lo que el checkout enumera al comprador.

Contrato 5 — Montaje de cada módulo (lo llama `main.go`, dueño: Agustín)

Cada módulo expone exactamente esta función, y `main.go` la llama una vez:

```
// backend/internal/adapters/rest/handlers/<módulo>/rutas.go
func Montar(r chi.Router, db *gorm.DB, cfg config.Config, mw *middleware.Middleware)
```

Contrato 6 — Migraciones

```
// backend/internal/adapters/db/postgresql/migrations/schema/<módulo>.go
func Migraciones<Módulo>() []*gormigrate.Migration
```

`registry.go` (Agustín) concatena las cuatro en orden fijo. Una migración mergeada **no se edita nunca**: se corrige con otra nueva.

Alcance recortado para el Parcial I

Entra: identidad con RBAC, catálogo con CRUD completo, carrito/checkout, pedido con copia histórica, reserva de stock por lotes, Mercado Pago en sandbox, transiciones de pedido, deploy.

No entra (vuelve para el Parcial II, 17/11): promociones, cupones, facturación con ARCA, estadísticas, favoritos, avisos de reposición, ventas externas, auditoría, WhatsApp, correo, geocodificación real con API de mapas, frontend.

Eso **se escribe en la sección 9 de `docs/specs.md`** antes del martes. Lo que no está recortado por escrito, en la defensa cuenta como funcionalidad prometida y no entregada.